

第四次实验：白盒测试实验

课程PPT参考：

[第5章 单元测试和集成测试.pdf - 坚果云 - 云盘|网盘|企业网盘|同步|备份|无限空间|免费网络硬盘|企业云盘](#)

实验背景

白盒测试是一种通过了解程序的内部结构和工作原理来进行测试的方式。在白盒测试中，测试人员需要通过对代码的深入理解，设计测试用例以覆盖代码的各个分支、条件、循环等。白盒测试能够发现隐藏在代码中的逻辑错误、未被测试到的路径以及性能瓶颈等问题。

Pytest实验

本实验将使用 pytest 对一个Python项目进行白盒测试，重点是通过代码覆盖率分析来检测代码的测试覆盖情况，并改进测试用例，以确保代码的质量。

下面会以一个pytest测试python计算器的简单示例说明如何使用pytest进行单元测试，并且如何检验测试覆盖率。

目标：

- 使用 pytest 编写和执行单元测试。
- 使用 pytest-cov 插件进行代码覆盖率分析。
- 检测并确保代码路径和分支得到了充分测试。

示例-实验步骤：

1. 工具与库准备：

- 安装 pytest 和 pytest-cov 插件：

代码块

```
1 pip install pytest pytest-cov
```

2. 实验代码：

- 我们将使用一个简单的 Python 计算器项目，包含加法、减法、乘法和除法功能。
- **calculator.py**（计算器功能实现）

代码块

```
1 class Calculator:
2     def add(self, a, b):
3         return a + b
4     def subtract(self, a, b):
5         return a - b
6     def multiply(self, a, b):
7         return a * b
8     def divide(self, a, b):
9         if b == 0:
10             raise ValueError("Cannot divide by zero")
11         return a / b
```

上面的代码实现了一个简单的计算器类 Calculator，包含加法、减法、乘法和除法功能。除法功能需要处理除数为零的特殊情况。

3. 编写测试用例：

- 使用 pytest 编写测试用例，确保对每个方法进行充分的测试，包括正常情况和边界情况。
- **test_calculator.py**（测试用例）

代码块

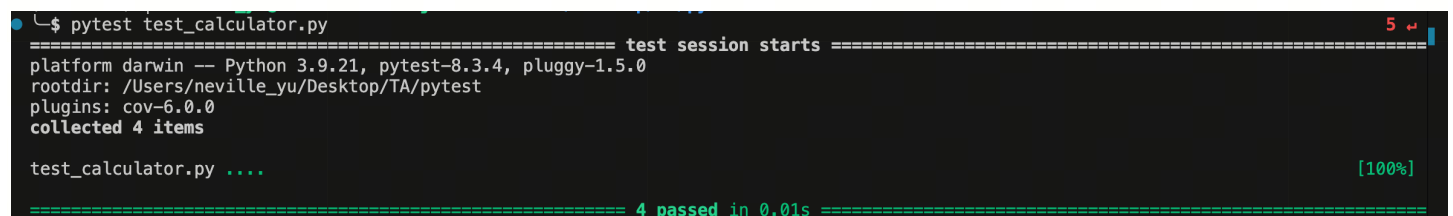
```
1 import pytest
2 from calculator import Calculator
3 创建 Calculator 实例
4 calc = Calculator()
5 def test_add():
6     assert calc.add(2, 3) == 5
7     assert calc.add(-1, 1) == 0
8     assert calc.add(0, 0) == 0
9 def test_subtract():
10     assert calc.subtract(5, 3) == 2
11     assert calc.subtract(0, 1) == -1
12     assert calc.subtract(-1, -1) == 0
13 def test_multiply():
14     assert calc.multiply(2, 3) == 6
15     assert calc.multiply(0, 1) == 0
16     assert calc.multiply(-1, 1) == -1
```

```
17 def test_divide():
18     assert calc.divide(6, 3) == 2
19     with pytest.raises(ValueError):
20         calc.divide(1, 0) # 除数为0时抛出异常
```

代码块

```
1 # Run this command
2 pytest test_calculator.py
```

执行上面这行命令后，你应该可以看到有类似下面的输出，表示我在0.01s内通过了4个测试案例（加、减、乘、除）



```
$ pytest test_calculator.py
===== test session starts =====
platform darwin -- Python 3.9.21, pytest-8.3.4, pluggy-1.5.0
rootdir: /Users/neville_yu/Desktop/TA/pytest
plugins: cov-6.0.0
collected 4 items

test_calculator.py .... [100%]

===== 4 passed in 0.01s =====
```

我们做一点小更改再来尝试一下，例如我在calculator中把add()函数的定义修改一下：

代码块

```
1 class Calculator:
2     def add(self, a, b):
3         """返回 a + b"""
4         return a - b # 这个就是我们在开发过程中不小心写错的代码
5 # 后省略
```

再次执行pytest，发现有一个测试用例未通过，并给出了具体原因（ $2-3 = -1$ 不等于5，所以发现代码错误）：

```
$ pytest test_calculator.py
===== test session starts =====
platform darwin -- Python 3.9.21, pytest-8.3.4, pluggy-1.5.0
rootdir: /Users/neville_yu/Desktop/TA/pytest
plugins: cov-6.0.0
collected 4 items

test_calculator.py F... [100%]

===== FAILURES =====
test_add
def test_add():
    assert calc.add(2, 3) == 5
E       assert -1 == 5
E       + where -1 = add(2, 3)
E       + where add = <calculator.Calculator object at 0x102955f40>.add

test_calculator.py:9: AssertionError
===== short test summary info =====
FAILED test_calculator.py::test_add - assert -1 == 5
===== 1 failed, 3 passed in 0.02s =====
```

4. 使用 pytest-cov 进行代码覆盖率分析：

使用 pytest-cov 插件来生成代码覆盖率报告，确保测试用例覆盖了代码中的所有重要路径。

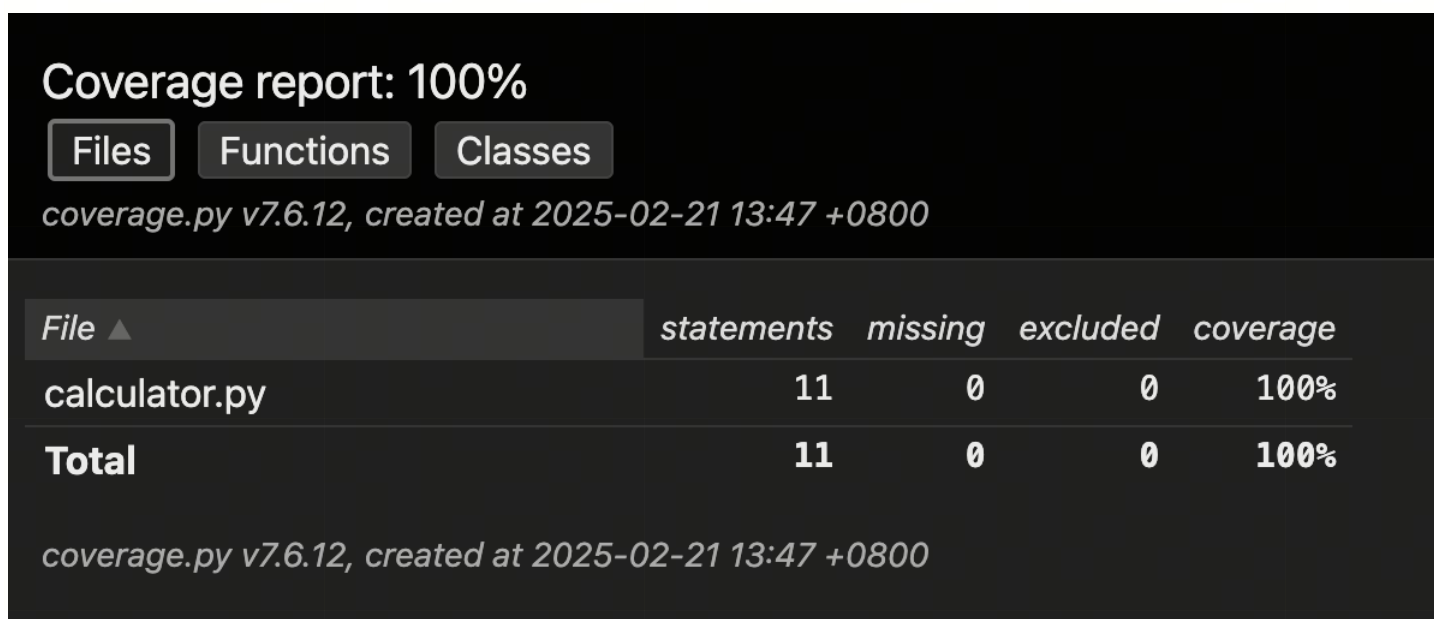
执行以下命令来运行 pytest 并生成覆盖率报告：

代码块

```
1  pytest --cov=calculator --cov-report=html
```

这里的 `--cov=calculator` 表示我们要测试 `calculator.py` 文件中的代码，`--cov-report=html` 表示生成 HTML 格式的覆盖率报告，报告将保存在 `htmlcov` 目录中。

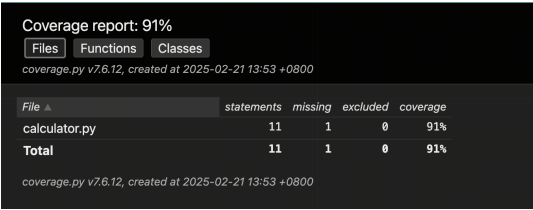
我们可以在浏览器中打开 `htmlcov/index.html` 这个文件，可以发现就是相关的测试信息，表示我们 100% 覆盖了所有代码：



🐞 那我们再做一点小修改，我们在test_calculator.py这个文件中把test_add()函数注释掉，然后再执行一次命令

代码块

```
1  # def test_add():
2  #     assert calc.add(2, 3) == 5
3  #     assert calc.add(-1, 1) == 0
4  #     assert calc.add(0, 0) == 0
```



File	function	statements	missing	excluded	coverage
calculator.py	Calculator.add	1	1	0	0%
calculator.py	Calculator.subtract	1	0	0	100%
calculator.py	Calculator.multiply	1	0	0	100%
calculator.py	Calculator.divide	3	0	0	100%
calculator.py	(no function)	5	0	0	100%
Total		11	1	0	91%

不难发现，Calculator.add的覆盖率变为0%了，这意味着测试过程中有一部分的代码没有测试到，所以需要额外编写测试函数进行测试。

因此，我们可以从这个页面检查测试用例的覆盖情况，确保每个分支、条件和循环都得到了充分的测试。

5. 优化测试用例：

根据覆盖率报告分析未覆盖的路径，修改或补充测试用例，确保所有代码路径和边界条件都被测试到。例如，如果发现 divide 方法中的除数为零的情况未被测试覆盖，学生可以根据报告补充相关测试用例。

测试目标：

请同学们仿照上面对python计算器的单元测试，对下面的todo_list.py文件也是用pytest等工具进行测试，并撰写实验报告。

代码块

```
1  class Todo:
2      def __init__(self, name, completed=False):
3          self.name = name
4          self.completed = completed
5
6      def mark_completed(self):
7          self.completed = True
8
9      def __str__(self):
10         return f'{self.name} - {'Completed' if self.completed else 'Pending'}'
```

```

11
12 class TodoList:
13     def __init__(self):
14         self.todos = []
15
16     def add_task(self, name):
17         if not name:
18             raise ValueError("Task name cannot be empty")
19         self.todos.append(Todo(name))
20
21     def remove_task(self, name):
22         task_to_remove = None
23         for task in self.todos:
24             if task.name == name:
25                 task_to_remove = task
26                 break
27         if task_to_remove:
28             self.todos.remove(task_to_remove)
29         else:
30             raise ValueError(f"Task with name '{name}' not found")
31
32     def mark_task_completed(self, name):
33         for task in self.todos:
34             if task.name == name:
35                 task.mark_completed()
36                 return
37         raise ValueError(f"Task with name '{name}' not found")
38
39     def get_all_tasks(self):
40         return [str(task) for task in self.todos]

```

Mock实验

为什么使用Mock🤖

Mock在英文中是“模仿”的意思，在本次实验的场景下，我们便使用Mock来模仿外部依赖模块。

虽然上面的pytest工具可以进行正确的白盒测试，但是在真实的软件开发中，很多模块会依赖外部系统或服务（如数据库、API调用等），**直接使用pytest测试这些模块可能会受到外部依赖的影响**（例如外部网络相应特别慢时会导致测试时间过长，或者一些特定的边界条件难以在真实情况中再现）。

于是，我们可以使用mock工具模拟这些外部服务的行为，在测试过程中避免实际调用外部服务。

目标：

- 使用mock模仿外部依赖服务，用pytest进行白盒测试

示例-实验步骤：

1. 工具与库准备：

代码块

```
1 pip install pytest-mock
2 pip install requests
```

2. 实验代码：

代码块

```
1  # calculator.py
2  class Calculator:
3      def add(self, a, b):
4          """返回 a + b"""
5          return a + b
6
7      def subtract(self, a, b):
8          """返回 a - b"""
9          return a - b
10
11     def multiply(self, a, b):
12         """返回 a * b"""
13         return a * b
14
15     def divide(self, a, b):
16         """返回 a / b, 如果除数为 0, 抛出异常"""
17         if b == 0:
18             raise ValueError("Cannot divide by zero")
19         return a / b
20
21  # data_processor.py
22  import requests
23
24  class DataProcessor:
25      def __init__(self, calculator):
26          self.calculator = calculator
27
28      def fetch_data(self, url):
29          """从指定URL获取数据"""
```

```

30         response = requests.get(url)
31         if response.status_code != 200:
32             raise ConnectionError("Failed to fetch data from the URL.")
33         return response.json()
34
35     def process_data(self, url):
36         """获取数据并计算总和"""
37         data = self.fetch_data(url)
38         total = 0
39         for item in data:
40             total = self.calculator.add(total, item)
41         return total

```

仍然是之前的计算器代码，但是在这里待测试的类是DataProcessor模块，它需要从制定URL获取数据(**依赖外部服务，需要mock模仿**)，并计算获取数据的总和。

3. 测试用例编写

代码块

```

1  # test_data_processor.py
2  import pytest
3  from calculator import Calculator
4  from data_processor import DataProcessor
5
6  @pytest.fixture
7  def calculator():
8      """Fixture: 提供Calculator实例"""
9      return Calculator()
10
11 @pytest.fixture
12 def data_processor(calculator):
13     """Fixture: 提供DataProcessor实例"""
14     return DataProcessor(calculator)
15
16 def test_process_data_success(mocker, data_processor):
17     """
18     测试process_data方法在成功获取数据时的行为
19     """
20     # Mock requests.get方法
21     mock_get = mocker.patch('data_processor.requests.get')
22     mock_response = mocker.MagicMock()
23     mock_response.status_code = 200
24     mock_response.json.return_value = [1, 2, 3, 4]
25     mock_get.return_value = mock_response

```



```

26
27     # 调用被测方法
28     result = data_processor.process_data("http://example.com/data")
29
30     # 验证结果
31     assert result == 10 # 1 + 2 + 3 + 4 = 10
32     mock_get.assert_called_once_with("http://example.com/data") # 验证URL是
    否正确调用
33
34 def test_process_data_failure(mock, data_processor):
35     """
36     测试process_data方法在无法获取数据时的行为
37     """
38     # Mock requests.get方法
39     mock_get = mock.patch('data_processor.requests.get')
40     mock_response = mock.MagicMock()
41     mock_response.status_code = 404
42     mock_get.return_value = mock_response
43
44     # 调用被测方法并捕获异常
45     with pytest.raises(ConnectionError, match="Failed to fetch data from
    the URL."):
46         data_processor.process_data("http://example.com/data")

```

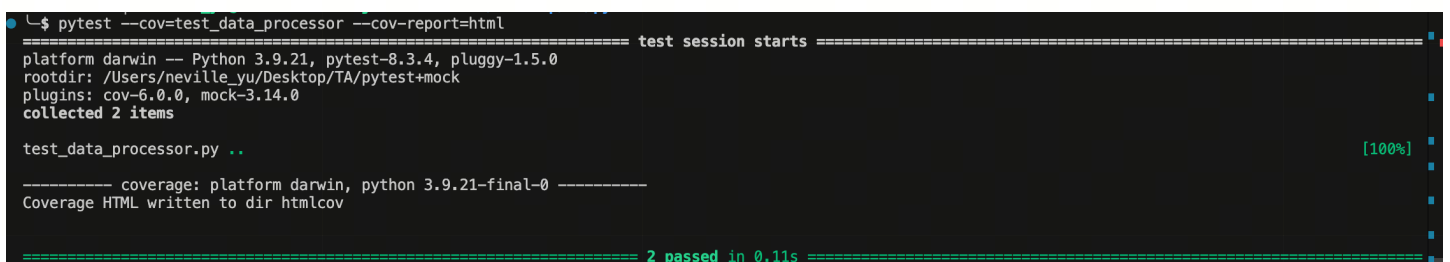
- 在上述测试中，`mock.patch` 替换了 `requests.get` 方法，避免了实际的网络请求。
- 通过 `MagicMock`，我们可以控制返回值（如 `status_code` 和 `json`），从而模拟各种场景。

4. 执行测试命令

代码块

```
1 pytest --cov=test_data_processor --cov-report=html
```

正确的结果如下所示：



```

$ pytest --cov=test_data_processor --cov-report=html
===== test session starts =====
platform darwin -- Python 3.9.21, pytest-8.3.4, pluggy-1.5.0
rootdir: /Users/neville_yu/Desktop/TA/pytest+mock
plugins: cov-6.0.0, mock-3.14.0
collected 2 items

test_data_processor.py .. [100%]

----- coverage: platform darwin, python 3.9.21-final-0 -----
Coverage HTML written to dir htmlcov

===== 2 passed in 0.11s =====

```

对应的html文件如下：

Coverage report: 100%
Files Functions Classes
coverage.py v7.6.12, created at 2025-04-09 16:13 +0800

File ▲	function	statements	missing	excluded	coverage
test_data_processor.py	calculator	1	0	0	100%
test_data_processor.py	data_processor	1	0	0	100%
test_data_processor.py	test_process_data_success	8	0	0	100%
test_data_processor.py	test_process_data_failure	6	0	0	100%
test_data_processor.py	(no function)	9	0	0	100%
Total		25	0	0	100%

同样，我们再做一点小修改，我们仍然把add的a+b改为a-b。

```
# calculator.py

class Calculator:
    def add(self, a, b):
        """返回 a + b"""
        return a - b
```

错误的结果如下所示：
这样就不是1+2+3+4，而是-1-2-3-4=-10，于是测试发现出错

```
===== short test summary info =====
FAILED test_data_processor.py::test_process_data_success - assert -10 == 10
===== 1 failed, 1 passed in 0.12s =====
```

对应的html文件如下：

Coverage report: 96%
Files Functions Classes
coverage.py v7.6.12, created at 2025-04-09 16:13 +0800

File ▲	function	statements	missing	excluded	coverage
test_data_processor.py	calculator	1	0	0	100%
test_data_processor.py	data_processor	1	0	0	100%
test_data_processor.py	test_process_data_success	8	1	0	88%
test_data_processor.py	test_process_data_failure	6	0	0	100%
test_data_processor.py	(no function)	9	0	0	100%
Total		25	1	0	96%

测试目标：

代码块

```
1  # weather_service.py
2  import requests
3
4  class WeatherService:
5      def __init__(self, api_key):
6          """
7          初始化WeatherService对象。
8          :param api_key: 外部天气API的访问密钥。
9          """
10         self.api_key = api_key
11
12     def fetch_weather_data(self, city):
13         """
14         调用外部天气API获取指定城市的天气数据。
15         :param city: 城市名称。
16         :return: API返回的JSON数据。
17         :raises ConnectionError: 如果API请求失败。
18         """
19         url = f"https://api.weatherapi.com/v1/current.json?key=
{self.api_key}&q={city}"
20         response = requests.get(url)
21         if response.status_code != 200:
22             raise ConnectionError(f"Failed to fetch weather data for {city}.
Status code: {response.status_code}")
23         return response.json()
24
25     def get_weather_summary(self, city):
26         """
27         获取指定城市的天气摘要信息。
28         :param city: 城市名称。
29         :return: 包含温度和湿度的天气摘要信息。
30         :raises ValueError: 如果API返回的数据格式不正确。
31         """
32         try:
33             data = self.fetch_weather_data(city)
34             temperature = data["current"]["temp_c"]
35             humidity = data["current"]["humidity"]
36             return {
37                 "city": city,
38                 "temperature": temperature,
39                 "humidity": humidity
40             }
41         except KeyError:
```

```
raise ValueError("Invalid data format received from the weather  
API.")
```

实验报告要求

完成实验后需要提交一份详细的实验报告，报告应包含以下内容：

1. 实验目的：

- 介绍白盒测试的基本原理和实验的目标。

2. 实验环境：

- 记录实验所使用的工具和版本，包括 Python 版本、pytest 版本、pytest-cov 插件版本等。

3. 实验内容：

- 描述针对两个测试目标是如何编写测试代码。

4. 测试过程：

- 记录测试用例的编写过程，特别是如何设计测试用例来覆盖不同的代码路径和边界条件。
- 描述如何使用 pytest 和 pytest-cov 运行测试并生成覆盖率报告。

5. 实验结果：

- 展示覆盖率报告，分析哪些代码路径被覆盖，哪些未被覆盖。
- 讨论是否有遗漏的测试路径，并优化测试用例以补充这些遗漏。

6. 实验总结：

- 总结白盒测试的优势，讨论通过白盒测试可以发现哪些潜在的问题。
- 反思如何将白盒测试应用到实际开发中，提高软件的可靠性和质量。